Agave v4.4

Security Assessment

Prepared for
Anza

Prepared by
Otter Audits, LLC

Prepared on
October 30th, 2026

Stepan Fedotov trixter@osec.io
Gabriel Schneider gbrls@osec.io

Table of Contents

1. Executive summary 2
1.1. Overview 2
1.2. Key findings 2
2. Scope 3
2.1. Transaction execution (SVM/runtime) 3
2.2. Staking, rewards & commission 3
2.3. Transaction scheduling & banking 3
2.4. Program loading & execution (sBPF) 3
2.5. Consensus & voting (Tower BFT) 3
3. Findings 4
3.1. Findings summary 4
4. Advisories 5
OS-AGV-ADV-00 ● `StakesCache` retains oversized account allocations 6
OS-AGV-ADV-01 ● Unguarded Alpenglow `ProgramCache` race 7
OS-AGV-ADV-02 ● EBPP relatching inflates `ProgramCache` 8
5. Suggestions 9
OS-AGV-SUG-00 ● Version-skewed final-slot deployment causes divergence 10
A. Vulnerability rating scale 11
B. Procedure 12

© 2026 Otter Audits LLC. All Rights Reserved.

1. Executive summary

1.1. Overview

Anza engaged OtterSec to conduct a security assessment of the agave v4.4 release (`v4.3.0..v4.4`). This assessment was conducted between September 24th and October 26th, 2026. For information on our auditing methodology, refer to Appendix B.

1.2. Key findings

We produced 4 finding throughout this audit engagement.

In particular, PR#15398's shallow cloning preserves oversized account-data allocations in `StakesCache` , enabling rent-underpriced memory amplification. An attacker can create thousands of small vote accounts retaining 10 MiB capacities, potentially exhausting validator memory with mostly recoverable capital (● OS-AGV-ADV-00). Additioanlly, Alpenglow's Block Creation Loop can write to the `ProgramCache` before its bank is visible in `BankForks` , bypassing Bank quiescence and racing with `ReplayStage` cleanup. A malicious leader could exploit this to create inconsistent cache state and cause consensus divergence (● OS-AGV-ADV-01). Furthermore, repeated EBPP relatching creates fresh pointer-identified runtime environments, causing equivalent programs to be requeued and recompiled instead of reused. This can inflate the `ProgramCache` , slow replay, and eventually cause an OOM-based denial of service (● OS-AGV-ADV-02).

Lastly, during a runtime-feature activation, Agave 4.3 may reject a final-slot program deployment that 4.4 accepts due to different deployment-environment selection. This version-dependent result can cause divergence in mixed-version clusters (● OS-AGV-SUG-00).

© 2026 Otter Audits LLC. All Rights Reserved.

2. Scope

The source code was delivered to us in a Git repository at `https://github.com/anza-xyz/agave`.

This audit was performed against the following PR's:

2.1. Transaction execution (SVM/runtime)

PR#13860, PR#14123, PR#14518, PR#14549, PR#14567, PR#14591, PR#14629, PR#14635, PR#14636, PR#14644, PR#14685, PR#14689, PR#14695, PR#14736, PR#14741, PR#14750, PR#14751, PR#14784, PR#14822, PR#14825, PR#14827, PR#14834, PR#14838, PR#14846, PR#14864, PR#14868, PR#14871, PR#14884, PR#14945, PR#14966, PR#14978, PR#14979, PR#14997, PR#15001, PR#15005, PR#15024, PR#15028, PR#15030, PR#15038, PR#15047, PR#15051, PR#15057, PR#15076, PR#15095, PR#15102, PR#15118, PR#15121, PR#15127, PR#15134, PR#15136, PR#15177, PR#15220, PR#15221, PR#15247, PR#15281, PR#15284, PR#15312, PR#15321, PR#15323, PR#15346, PR#15365, PR#15366, PR#15390, PR#15391, PR#15393, PR#15394, PR#15398, PR#15400, PR#15406.

2.2. Staking, rewards & commission

PR#14394, PR#14497, PR#14946, PR#14994, PR#15029, PR#15078, PR#15130, PR#15169, PR#15242, PR#15278, PR#15430.

2.3. Transaction scheduling & banking

PR#14356, PR#14542, PR#14584, PR#14654, PR#14661, PR#14662, PR#14668, PR#14739, PR#14740, PR#14836, PR#14869, PR#14915, PR#15016, PR#15018, PR#15056, PR#15059, PR#15072, PR#15163, PR#15313, PR#15350.

2.4. Program loading & execution (sBPF)

PR#14392, PR#14630, PR#14690, PR#14712, PR#14774, PR#14776, PR#14819, PR#14820, PR#14821, PR#14847, PR#14850, PR#14877, PR#14881, PR#14882, PR#14883, PR#14907, PR#14927, PR#14929, PR#14939, PR#14951, PR#14962, PR#14969, PR#14989, PR#14990, PR#15071, PR#15092, PR#15135, PR#15183, PR#15272, PR#15402.

2.5. Consensus & voting (Tower BFT)

PR#15026, PR#15120, PR#15286, PR#15324.

© 2026 Otter Audits LLC. All Rights Reserved.

3. Findings

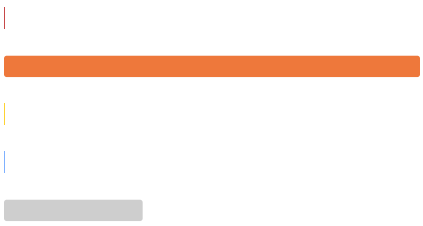
Findings were rated using the criteria in Appendix A.

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: `OS-AGV-ADV-*` for advisories and `OS-AGV-SUG-*` for suggestions.

3.1. Findings summary

Severity	Count
Critical	0
High	3
Medium	0
Low	0
Informational	1



© 2026 Otter Audits LLC. All Rights Reserved.

4. Advisories

Here, we present a technical analysis of the 3 vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

ID	Severity	Status	Description
OS-AGV-ADV-00	● High	✓ Resolved	`StakesCache` retains oversized account allocations
OS-AGV-ADV-01	● High	✓ Resolved	Unguarded Alpenglow `ProgramCache` race
OS-AGV-ADV-02	● High	✓ Resolved	EBPP relatching inflates `ProgramCache`

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix A.

Vulnerability rating scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of ✓ Resolved, ○ Ack'd, or × To-do.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process

© 2026 Otter Audits LLC. All Rights Reserved.